CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL 3º CURSO

UNIVERSIDAD POLITÉCNICA DE MADRID

APRENDIZAJE AUTOMÁTICO

PRACTICA DE SUPERVISADO

10/12/2025

DIEGO JOSÉ GARCÍA CALLEJAS

dj.garcia.callejas@alumnos.upm.es

HÉCTOR FERNÁNDEZ CANO

hector.fernandez@alumnos.upm.es

ÍNDICE

ÍNDICE 1
1. Introducción 1
2. Análisis de las variables 1
3. K-Nearest Neighbors (KNN) 3
4. Support Vector Machine (SVM) 4
5. Regresión logística 6
6. Árbol de decisión 7
7. Mejor modelo 9
8. Conclusión 10

1. Introducción

Este trabajo abordará un estudio de aprendizaje automático supervisado aplicado en nuestro caso a la predicción del tipo de lanzamiento en beisbol a partir de características relacionadas con la física del lanzamiento, características del jugador y contexto de los partidos . Para ello utilizaremos el conjunto de datos MLB Postseason 2025 Pitch-by-Pitch Data extraido de la página: https://www.kaggle.com/datasets/janus137/mlb-postseason-2025-pitch-by-pitch-data

Dadas las instrucciones del trabajo , a lo largo de la memoria se procederá al desarrollo y comparación de distintos modelos supervisados: regresión logística, árboles de decisión, k-NN y máquinas de vectores soporte (SVM). De cada uno de ellos se evaluará su rendimiento mediante validación cruzada y métricas adecuadas para responder a las distintas cuestiones .

El objetivo final de nuestro trabajo es identificar el modelo que mejor predice la categoría de tipo de lanzamiento y analizar los factores más influyentes en dicha clasificación, comparando con los otros modelos y extrayendo conclusiones acordes.

Esta memoria se complementará con el cuaderno jypiter ya que debido al límite máximo de páginas, nos veremos forzados a prescindir en esta memoría de algunos datos.

2. Análisis de las variables

El apartado de análisis y tratamientos de las variables no será evaluado en esta entrega, sin embargo, creemos necesario para el entendimientos de todo el trabajo, comentar brevemente el preprocesado.

En primer lugar, es importante destacar que el conjunto de datos inicial consta de 95 variables y 14.096 observaciones, por motivos de tiempos de ejecución y variables que no aportaban información, hemos decidido reducir dicho conjunto a 29 variables, donde todas ellas aportan información relevante y 5000 observaciones, para reducir dicho tiempo.

En segundo lugar, las 29 variables predictoras usadas por nuestro modelo son las siguientes, las cuales están explicadas en el cuaderno por temas de extensión

- Balls, Strikes, Outs_when_up, On_1b, On_2b, On_3b, Zone, Plate_x, Plate_z, Pfx_x, Pfx_z, Release_speed, Effective_speed, Release_pos_x, Release_pos_y, Release_pos_z, Vx0, Vy0,

Vz0, Ax, Ay, Az, Release_spin_rate, Spin_axis, Release_extension, P_throws, Stand, Sz_bot y Sz_top

Y en tercer lugar, la variable objetivo, es decir, la que el modelo intenta estimar es la siguiente:

La variable **pitch_type** indica el tipo de lanzamiento que ha realizado el pitcher. Cada categoría corresponde a un estilo distinto, con diferencias en velocidad, movimiento y efecto. En este dataset aparecen los siguientes diez tipos de lanzamientos:

- **CH (Changeup):** lanzamiento más lento que la recta, pensado para engañar al bateador haciéndole creer que la pelota viene más rápido de lo que realmente va.
- **CU (Curveball):** lanzamiento que hace una caída bastante visible justo antes de llegar al home, utilizado para sorprender al bateador.
- **FC (Cutter):** recta que se mueve ligeramente hacia un lado al final, lo que dificulta un buen contacto.
- **FF (Four-Seam Fastball):** la recta clásica y el lanzamiento más rápido. Va muy directa y es la más común entre los pitchers.
- **FS (Splitter):** lanzamiento que parece una recta al salir de la mano, pero cae de forma repentina al acercarse al bateador.
- **KC (Knuckle Curve):** una versión más marcada de la curva, con un quiebre más fuerte y definido.
- **SI (Sinker):** recta que "se hunde" al final, bajando más de lo esperado y provocando muchos batazos por el suelo.
- **SL (Slider):** lanzamiento con un movimiento lateral rápido que se aleja del bateador o se mete hacia él, según el lado del pitcher.
- **ST (Sweeper):** una variante del slider con un movimiento lateral todavía más amplio, arrastrándose hacia un lado de forma llamativa.
- **SV (Slurve):** mezcla entre slider y curveball; combina un poco de caída con movimiento lateral.

Por último, se llevó a cabo el preprocesamiento del dataset con el fin de preparar las variables para su uso en los modelos de clasificación. Las tareas realizadas fueron las siguientes:

1. **Transformación de variables categóricas a numéricas**
   Algunos de los modelos empleados (SVM, KNN y Regresión Logística) requieren que todas las variables sean numéricas. Las variables categóricas seleccionadas en este proyecto son binarias, por lo que no fue necesario utilizar OneHotEncoder. La transformación se realizó manualmente:
   p_throws y stand: toman valores L y R. Se mapean como:
   L → 0
   R → 1
   on_1b, on_2b, on_3b: indican si hay un corredor en cada base. Los valores nulos en estas columnas no significan pérdida de datos, sino que la base está vacía, por lo que se transformaron de la siguiente forma:
   Base vacía → 0
   Base ocupada → 1

2. **Tratamiento de valores nulos**
   Además de los nulos asociados a las bases, algunas variables presentaban valores vacíos

ocasionales, que simplemente indican ausencia de información en esa jugada. En estos casos, y dada su baja frecuencia, se decidió reemplazarlos por 0, manteniendo la coherencia con las transformaciones anteriores.

3. **División del dataset en entrenamiento y test**
   Se aplicó la división estándar del 80% de los datos para entrenamiento (incluyendo validación mediante Cross-Validation) y el 20% restante para test. Esto garantiza que el rendimiento final se evalúa sobre datos que el modelo no ha visto durante el entrenamiento.

4. **Escalado de las variables numéricas**
   Tras la separación Train/Test, se procedió al escalado para evitar fuga de información. El método empleado fue StandardScaler, que estandariza cada variable restando la media y dividiendo por la desviación estándar. Para lograr una aproximación más profesional,decidimos incorporarlo dentro de un Pipeline, de manera que en la validación cruzada el scaler se ajustaba únicamente con los datos de entrenamiento de cada fold y posteriormente se aplicaba al correspondiente conjunto de validación. Una vez seleccionado el mejor modelo mediante GridSearchCV, el conjunto de test se era escalado exclusivamente con los parámetros aprendidos del conjunto de entrenamiento.

3. K-Nearest Neighbors (KNN)

En primer lugar, se analizaron los hiperparámetros a probar con el fin de determinar automáticamente la configuración del mejor modelo KNN. Para el número de vecinos (k), se estableció un rango entre 1 y 20. Este rango permite explorar desde modelos muy sensibles al detalle hasta otros que permiten una mayor flexibilidad, ofreciendo una variedad suficiente sin provocar un incremento excesivo en el tiempo de ejecución durante la validación.

Dado que KNN basa su funcionamiento en el cálculo de distancias entre observaciones, se incluyeron dos métricas ampliamente utilizadas: la distancia Euclídea y la distancia Manhattan.

Todas las combinaciones posibles entre los valores de k y las métricas de distancia se evaluaron mediante la función GridSearchCV, aplicando Cross-Validation con 10 folds.
La métrica utilizada para optimizar el rendimiento fue el F1 macro, ya que el objetivo principal del estudio es obtener un rendimiento equilibrado entre todas las clases, independientemente de su tamaño o frecuencia.

El resultado del proceso anterior nos devuelve el modelo que obtiene el mejor F1 macro, siendo este 0.79433. Toda esta información se puede consultar directamente en el código, dentro del objeto best_knn, donde aparecen los hiperparámetros seleccionados automáticamente por GridSearchCV para generar el mejor modelo KNN. A partir de ese resultado, se observa que el número de vecinos elegido es k = 1 y que la métrica de distancia utilizada es la distancia Manhattan, ya que es la combinación que ofreció el mejor comportamiento en la validación cruzada.

A continuación, en el código se muestra una gráfica donde se comparan los distintos valores de k tanto para la distancia euclídea como para la distancia Manhattan, junto con el F1 macro obtenido en cada caso. En la gráfica se puede ver que la distancia Manhattan es la que mejores resultados da y que el pico más alto se alcanza con k = 1, coincidiendo con el modelo seleccionado anteriormente por GridSearchCV.

4. Support Vector Machine (SVM)

En primer lugar, se analizaron todos los posibles hiperparámetros que se van a probar, con el objetivo de encontrar la mejor combinación que permita obtener el mejor modelo SVM. Se ha utilizado una lista de penalizaciones C con los valores 0.1, 1, 10 y 100, de forma que se incluyan opciones desde una penalización pequeña, que favorece una mayor generalización, hasta otras más altas, que producen un ajuste más estricto del modelo.

En cuanto al kernel, se seleccionaron linear y rbf. El primero es adecuado cuando los datos pueden separarse de forma más o menos lineal, mientras que el segundo permite modelar fronteras de decisión no lineales. Estos dos kernels cubren las dos posibilidades principales a la hora de separar los datos, por lo que se consideró suficiente trabajar solo con ellos. Además, añadir más opciones habría incrementado notablemente el tiempo de ejecución.

Al incluir un kernel no lineal es necesario indicar el parámetro γ, el cual ajusta la flexibilidad de la frontera de decisión. Para ello hemos usado dos opciones: scale y auto. La opción scale tiene en cuenta tanto el número de variables como la varianza de los datos, lo que normalmente da lugar a valores de γ más pequeños y, por tanto, a fronteras de decisión más suaves y con mayor capacidad de generalización. Por otro lado, auto solo utiliza el número de variables, lo que tiende a generar valores de γ más altos que scale y produce fronteras más complejas, ajustándose más al conjunto de entrenamiento.

Para terminar con los hiperparámetros, se tiene en cuenta class_weight, que indica el peso de las clases durante el entrenamiento. Se han probado dos opciones: None, donde todas las clases tienen el mismo peso, de modo que si el conjunto de datos está desbalanceado las clases más frecuentes influyen más en el modelo; y balanced, donde las clases con menos observaciones reciben un mayor peso para compensar este desbalanceo y evitar que el modelo tienda a favorecer a las clases mayoritarias. Aunque podría esperarse que la opción balanced sea siempre la más adecuada, en algunos casos el modelo ya es capaz de manejar el desbalanceo a partir del resto de hiperparámetros, por lo que también se incluye la opción None para no descartar esa posibilidad.

Finalmente, todas las combinaciones posibles de los hiperparámetros comentados se evalúan mediante validación cruzada utilizando la función GridSearchCV, con 10 folds. Esta función devuelve el mejor modelo, es decir, la combinación de hiperparámetros que obtiene el mayor rendimiento según la métrica seleccionada. En este caso, se ha utilizado f1_macro, ya que esta métrica tiene en cuenta la presencia de clases con menos observaciones y ofrece una evaluación más equilibrada. El mejor modelo obtenido alcanza un f1_macro en validación de 0.81425.

A continuación se analiza el mejor modelo obtenido y los hiperparámetros utilizados para conseguirlo. Esta información se puede consultar en el objeto best_svm, donde vemos que la penalización C es 10, un valor intermedio que permite un ajuste más preciso sin perder del todo la capacidad de generalización. El kernel elegido es rbf, lo que indica que la frontera de decisión probablemente sea curva, ya que los datos no parecen separarse bien con un límite lineal.

Como se comentó antes, en los kernels no lineales es necesario definir un valor para γ, que ajusta la forma de la frontera. En este caso, el mejor modelo utiliza γ= 'scale', por lo que la frontera será algo más suave, favoreciendo cierta generalización. Por último, class_weight aparece como None, lo que encaja con la idea mencionada previamente de que el modelo, con el resto de hiperparámetros, ya es capaz de manejar el desbalanceo sin necesidad de ajustar los pesos de las clases.

Si nos fijamos en la figura (Figure 6: Evaluation of hyperparameters) se muestra cómo evoluciona el f1_macro para los distintos valores de C y para los dos kernels utilizados. En el caso del kernel linear, se ve que el rendimiento apenas varía con los distintos valores de C y se mantiene alrededor de 0.68. Esto indica que, en este problema, una frontera lineal no consigue separar bien los datos, por lo que aumentar o reducir la penalización no tiene demasiado efecto.

En cambio, el kernel rbf muestra un comportamiento bastante diferente. Aquí el rendimiento sí mejora cuando aumentamos C, especialmente al pasar de C = 0.1 a C = 1, alcanzando su mejor resultado alrededor de C = 10. Tanto con γ= scale como con γ= auto la evolución es similar, aunque scale obtiene valores ligeramente superiores.

En definitiva, la gráfica permite ver claramente la diferencia entre ambos kernels: el lineal apenas cambia y se queda en un rendimiento bajo, mientras que rbf se adapta mucho mejor a los datos y mejora a medida que ajustamos C. Por tanto, rbf resulta más adecuado para este problema.

5. Regresión logística

Para abordar la regresión logística, al igual que en los otros modelos, analizamos los posibles hiper parámetros para poder determinar el mejor modelo. Para el parámetro 'C' el cual controla la intensidad con la que el modelo regulariza (inversa del coeficiente de penalización), probamos un amplio rango desde 0.001 hasta 1000, lo que nos permite probar modelos más simples y otros un poco más libres. En cuanto al modo de penalizar utilizamos 'L2', puesto que es la más estable y la más usada.

Respecto al método de optimización ('solver') probamos a utilizar dos que funcionan bien con multiclases y la penalización usada, los cuales son, 'newton-cg', el cual por su parte se basa en la segunda derivada y 'lbfgs', que también se basa en lo mismo, pero es más rápido y se utiliza en conjuntos de dato medianos.

Por último, decidimos comprobar si era necesario corregir clases desbalanceadas mediante el parámetro 'class_weight', usando dos posibles valores 'None', como su propio nombre indica no ajusta pesos o 'balanced' el cual ajusta pesos inversamente proporcionales a la frecuencia de cada clase.

Tras ejecutar el GridSearchCV observamos que el valor C óptimo devuelto es 300, lo cual índica que el modelo requiere un nivel de regularización bajo, para asi poder ajustarse a los datos. Respecto al método de optimización obtuvimos 'lbfgs' como el mejor, y en cuanto al peso de las clases, 'None'es la mejor opción, por lo que los pesos de cada clase se tratan según como están.

El modelo con estos parámetros obtuvo un 'f1_macro' medio de 0.67441 en validación, con un rango entre (0.63,0.74), reflejando un rendimiento estable, con algo de variabilidad, aspecto esperable en un problema de multiclase.

Con el mejor modelo, era turno de analizarlo mediante diferentes gráficas y métricas. Empezamos analizando la importancia que tiene cada característica, lo cual en este modelo viene determinado por el coeficiente de cada una. Tras graficarlo (Figure 8: Feature Importance (Logistic Regression)) pudimos determinar que las características físicas y mecánicas como pueden ser la velocidad o la posición de la bola en determinados momentos son más importantes que el propio contexto del juego (

ej: número de corredores en una base), lo cual tiene bastante sentido, ya que un lanzamiento es algo casi puramente físico. Y aunque el contexto del juego no tenga tanta importancia, se observa que puede que alguna variable si que influya algo más ('balls').

6. Árbol de decisión

Por último, pero no menos importante, investigamos el modelo de árbol de decisión. En este modelo decidimos investigar los siguientes hiper parámetros:

- 'min_samples_leaf': este parámetro establece el número mínimo de muestras que debe contener una hoja del árbol. Probamos distintos valores, desde 1 hasta 10. Los valores más pequeños generaría árboles más precisos pero puede que sobre ajustados, mientras que valores más grandes hacen que el modelo generalice más.
- 'min_samples_split': este parámetro determina cuántas muestras son necesarias para que un nodo se divida, siendo muy importante en la profundidad que tendrá el modelo. En nuestro caso hemos probado valores más pequeños como 2 o 5, que darían lugar a muchas ramificaciones y valores más pequeños como 10 o 20, que darían lugar a modelos más generales y compactos.
- 'max_depth': indica la profundidad máxima que tendrá el modelo. En nuestro caso utilizamos solo los parámetros 2 y 5, ya que quisimos controlar la complejidad,quitando el riesgo de tener un mayo sobreajuste
- 'criterion': este parámetro lo que determina es la métrica que utiliza el árbol de decisión para evaluar la calidad de una división en cada nodo. En nuestros casos utilizamos entropy y gini, para ver cual de ellos es mejor.
- 'ccp_alpha': este parámetro está relacionado con la poda. Evaluaremos valores desde 0, es decir, sin uso de poda, hasta, 0.001, que permitiría controlar aquellas ramas que no aportan información y solo generan sobreajuste del modelo
- max_features: nos permite controlar el número de variables que utiliza el árbol para la decisión. Hemos probado None, el cual utiliza todas ellas (valor por defecto), sqrt, que se basa en la raíz del número de variables, y log2, que se basa en el logaritmo, lo cual reduce el número de variables usadas aún más.

Tras ejecutar el GridSearchCV observamos que los mejores parametros para el modelo son ccp_alpha=0.0005, lo cual indica que es necesario algo de poda, criterion='entropy' (se basa en teoría de la información), max_depth=5,max_feautures=None, luego utiliza todas las variables,min_samples_leaf=4 and min_samples_split='5' (más ramificaciones). Con estos criterios intuimos que obtendremos un modelo bastante interpretable, aunque lo averiguaremos más adelante

El modelo con estos parámetros obtuvo un 'f1_macro' medio de 0.617 en validación, con un rango entre (0.57,0.65), reflejando un rendimiento no muy bueno, peor que en los otros modelos, por lo que a lo mejor este tipo de modelo no se ajusta bien al problema planteado.

El procedimiento ahora es similar a los otros modelos, ya que debemos analizarlo con gráficos y métricas. Empezamos analizando la importancia de las variables, ya que a diferencia del modelo de regresión logística, aquí la importancia reside en cómo ayuda cada variable a reducir la impureza ,es decir,el número de clases distintas que hay en cada nodo. La gráfica (en el cuaderno) muestra como el árbol solo se basa en un conjunto pequeño de variables las cuales están relacionadas con las características físicas del lanzamiento. Seguramente al confiar tanto en tan pocas obtenemos peores resultados.

Tras este proceso, verificamos que no se produjera sobreajuste. Para ello comparamos el rendimiento del modelo en el conjunto de entrenamiento con el obtenido en el conjunto de Cross-Validation utilizando la métrica F1_macro. La diferencia del resultado es de 0.02 aproximadamente por lo que podemos decir con total seguridad que no ocurre overfitting.

Finalmente, analizaremos la parte más importante de este modelo seguramente - la estructura del árbol- mediante la cual podremos analizar las reglas en las que se basa para separar, aunque con los pesos disponemos de una ligera intuición. El árbol resultante profundidad 5 con 31 nodos terminales:

- Primer nivel: la primera división se realiza en función de la variable con más peso vy0, es decir, la velocidad inicial hacia el plato, que se divide en lanzamientos rápidos y lanzamientos lentos, ya que los lanzamientos que no son tan rectos tendrán velocidad menor. Esta división entonces genera dos visiones en el segundo nivel.
    - Segundo nivel- lanzamientos rápidos: esta rama la regla de división se basa en pfx_z (la segunda variable más importante), la cual nos indica cuánto se desvía verticalmente el lanzamiento respecto a la trayectoria sin spin. Algunos lanzamientos como FF tienden a no bajar tanto en cambio otros sí como SI o FC. De nuevo, esta división genera otras dos divisiones
        - Tercer nivel - lanzamientos con mayor caída : en esta rama con lanzamientos rápidos y mayor caida se observa que hay un lanzamiento dominante (SI), el cual está intentando limpiar con el release_spin_rate, rotación al salir de la mano del lanzador. En los lanzamientos SI esta es menor
        - Tercer nivel - lanzamientos con menor caída : en los lanzamientos más rectos, rápidos y sin apenas caída podemos observar que la mayor parte de los lanzamientos son FF, por lo que al igual que en el caso anterior, intenta limpiar , en este caso con pfx_x
    - Segundo nivel- lanzamientos lentos : en la rama de los lanzamientos lentos, lo siguente que se tiene en cuenta es el release_spin_rate, que como dijimos anteriormente, es la rotación con la que sale la bola tras ser lanzada. Esto nos permitirá separar aquellos lanzamientos que son un cambio de velocidad (ejemplo:CH), de aquellos más curvos (ejemplo:CU)
        - Tercer nivel - lanzamientos con menor giro : aquellos lanzamientos lentos con menor giro domina casi el lanzamiento CH, por lo que se mira spin_axis, es decir, el eje de rotación, para ver que la bola realiza el movimiento típico de este tipo de lanzamiento
        - Tercer nivel - lanzamientos con mayor giro : en este tipo de lanzamiento miramos ahora pfx_z,la cual nos indica cuánto se desvía verticalmente el lanzamiento respecto a la trayectoria sin spin, para ver si la bola cae mucho, típico de lanzamientos como CU o cae menos, típico en SL

7. Mejor modelo

Una vez finalizado el análisis de todos los modelos , es turno de elegir el mejor de ellos, para lo cual vamos a generar una tabla que respalde nuestra decisión
A continuación se adjunta una gráfica, a modo de resumen mostrando el f1 macro obtenido para los distintos modelos probados en Cross-Validation.

Modelos	F1_Macro
KNN	0.79433
SVM	0.81425
Regresión logística	0.67441
Árbol de decisión	0.61780

A partir de estos resultados, observamos que el modelo que obtiene el mejor rendimiento en validación cruzada es SVM, con un f1 macro superior al del resto de técnicas. Es importante destacar que estos valores corresponden a los mejores modelos encontrados tras optimizar sus hiperparámetros. Por tanto, seleccionamos SVM como modelo final.

Posteriormente, evaluamos el comportamiento de estos modelos en el conjunto de test. Aunque esta comparación es únicamente informativa, nos permite confirmar que el rendimiento del SVM no produce sobreajuste.

Modelo	Accuracy	Precision	Recall	f1 macro
SVM	0.87987	0.84974	0.74884	0.76726

El f1 macro del SVM aplicado sobre el conjunto de test disminuye aproximadamente un 5% respecto al obtenido en validación cruzada. Esta caída es moderada y entra dentro de lo esperable al evaluar sobre un conjunto de datos no visto previamente.

Al comprobar el f1 macro obtenido al aplicar el modelo sobre el conjunto de entrenamiento 0.9824, el cúal difiere en un 16% con el obtenido en Cross-Validation por lo que puede ser un indicio de que haya sobreajuste, sin embargo al ver los parámetros obtenidos en el mejor modelo no están en los extremos del param_grid, los kernels utilizados no son extremadamente complejos que puedan hacer que el modelo memorice por lo que hemos considerado que esta diferencia se debe a una situación derivada de los datos. Sin embargo, como la diferencia es grande, se puede considerar que haya sobreajuste, pero los modelos utilizados en esta práctica no ofrecen suficientes mecánismos para poder solucionarlo.

Al analizar el informe de clasificación del modelo SVM (Figure 15: *Classification report heatmap*), se observa que las clases con mayor número de muestras, como *FF*, presentan métricas altas. Esto era esperable, ya que disponer de suficientes ejemplos en el conjunto de entrenamiento facilita que el modelo aprenda correctamente los patrones característicos de estas clases.

Por el contrario, en clases con pocos datos, como *SV*, las métricas disminuyen de forma notable, especialmente la precisión. Esto indica que el modelo tiende a confundir otras clases con *Slurve*, debido a que no dispone de suficientes ejemplos para diferenciarlas adecuadamente.

Si pensamos en una posible aplicación práctica, como asistir al bateador en la interpretación del lanzamiento, este tipo de errores podría tener un impacto relevante. Una predicción incorrecta de

Slurve puede llevar al bateador a anticipar un movimiento del balón que realmente no se va a producir, lo que dificultará la toma de decisiones en el momento del swing.

Finalmente analizamos la matriz de confusión, en está matriz podemos ver un recuento de cuantas veces se predice correctamente o incorrectamente una clase. En ella se ve la misma situación que hemos comentado en el informe de clasificación donde el modelo funciona muy bien para clases las cuales tienen muchas observaciones, mientras que en las que menos observaciones tiene el rendimiento es peor, fallando más en su predicción.

8. Conclusión

Una vez finalizado este trabajo, hemos aprendido el significado y la importancia de muchos de los hiperparámetros utilizados en los distintos modelos, así como cómo sus ajustes pueden influir en el rendimiento final. También hemos sido capaces de descartar técnicas como SMOTE a partir del entendimiento de su funcionamiento y de las características específicas de nuestro dataset, lo cual resulta especialmente valioso, ya que no siempre es adecuado aplicar de forma automática todo lo visto en clase.

Otro aprendizaje relevante ha sido comprender el procedimiento profesional de un proyecto de aprendizaje automático. Entre estos aspectos destaca la correcta aplicación del escalado sin provocar fuga de datos, y la integración de todo el proceso en un pipeline dentro de cada fold de Cross-Validation. Esto nos ha permitido trabajar siguiendo prácticas adecuadas y reproducibles, similares a las empleadas en entornos reales.

Algunas mejoras que nos gustaría comentar son entre ellas la disposición de una cantidad mayor de aquellas clases menos frecuentes, ya que esto nos permitiría impulsar los modelos para que aprendiesen con mayor profundidad las estructuras. Otra mejora que nos hubiese gustado es aplicar otros tipos de modelos más complejos los cuales puede que se ajusten más a nuestros datos por su estructura compleja.

Para terminar, podemos afirmar que el flujo de trabajo seguido durante el desarrollo de esta práctica ha sido adecuado. La diferencia entre el F1-macro obtenido en la Cross-Validation (CV) y el obtenido en el conjunto de test es de aproximadamente un 5%, una variación pequeña y dentro de lo esperable. Esto confirma que el proceso aplicado ha sido el correcto.